

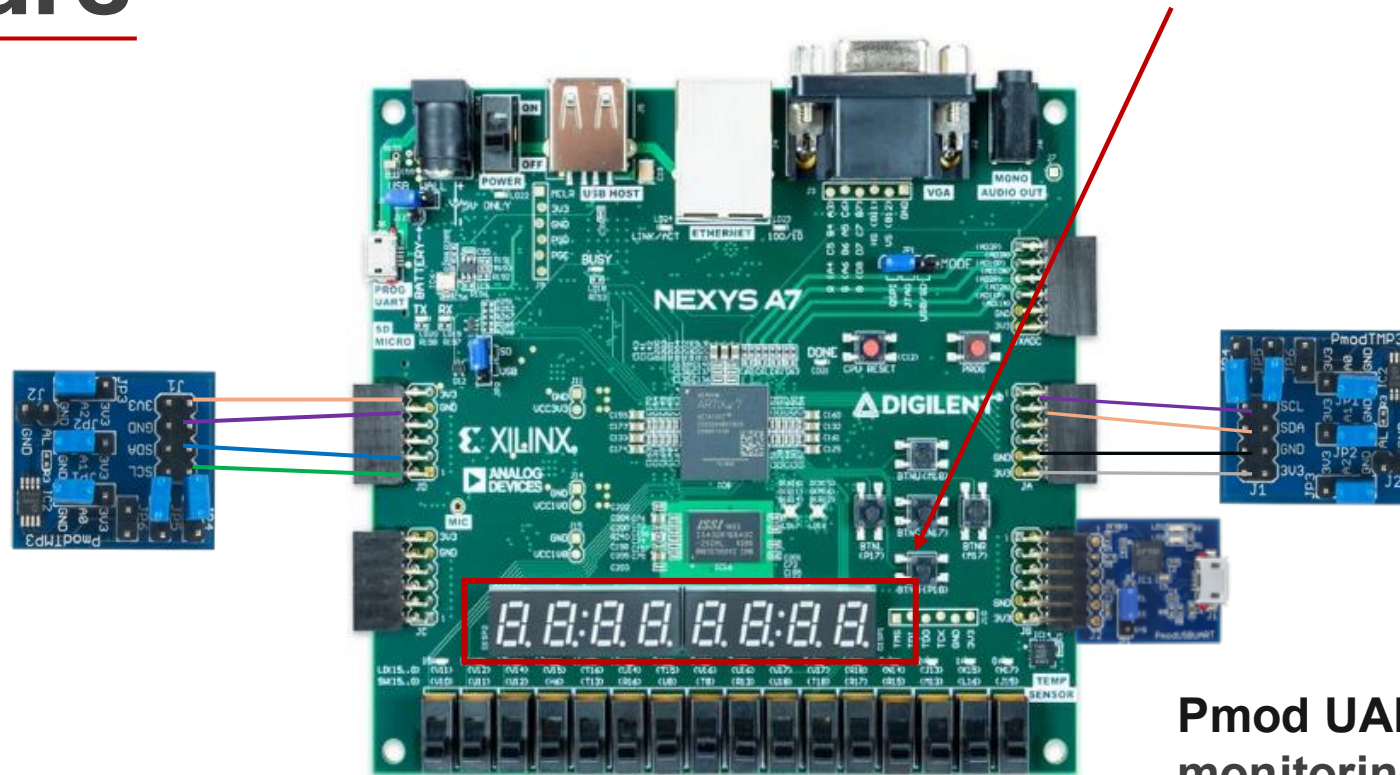
Functional Safety in Thermal Management Systems

Presented by

**Maria Anthonyos
Mohamad Abdo
Jonathan Bahry**

Hardware

The Seven segment display is used to indicate system states such as COOL and STOP

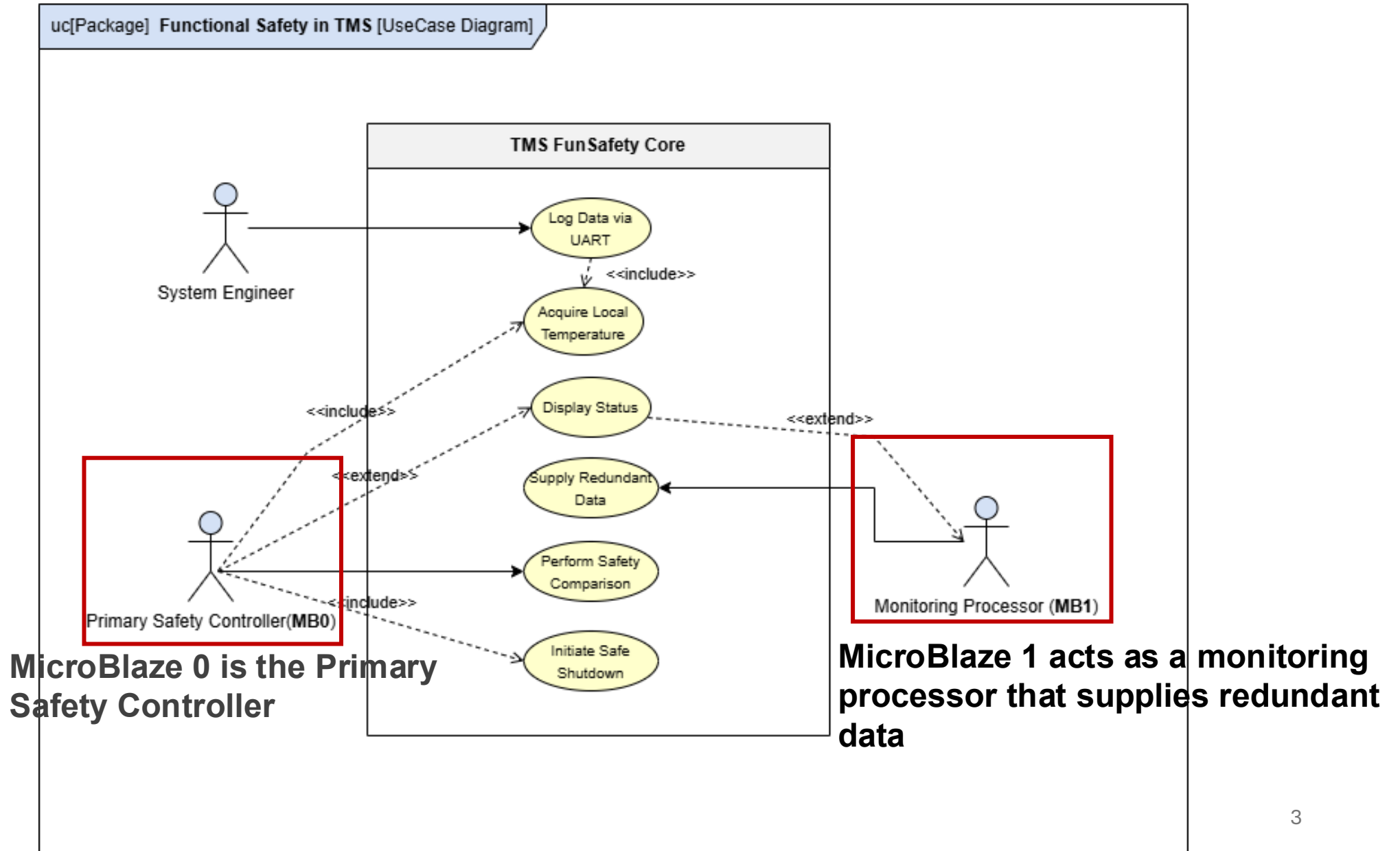


Pmod UART is used only for monitoring and debugging

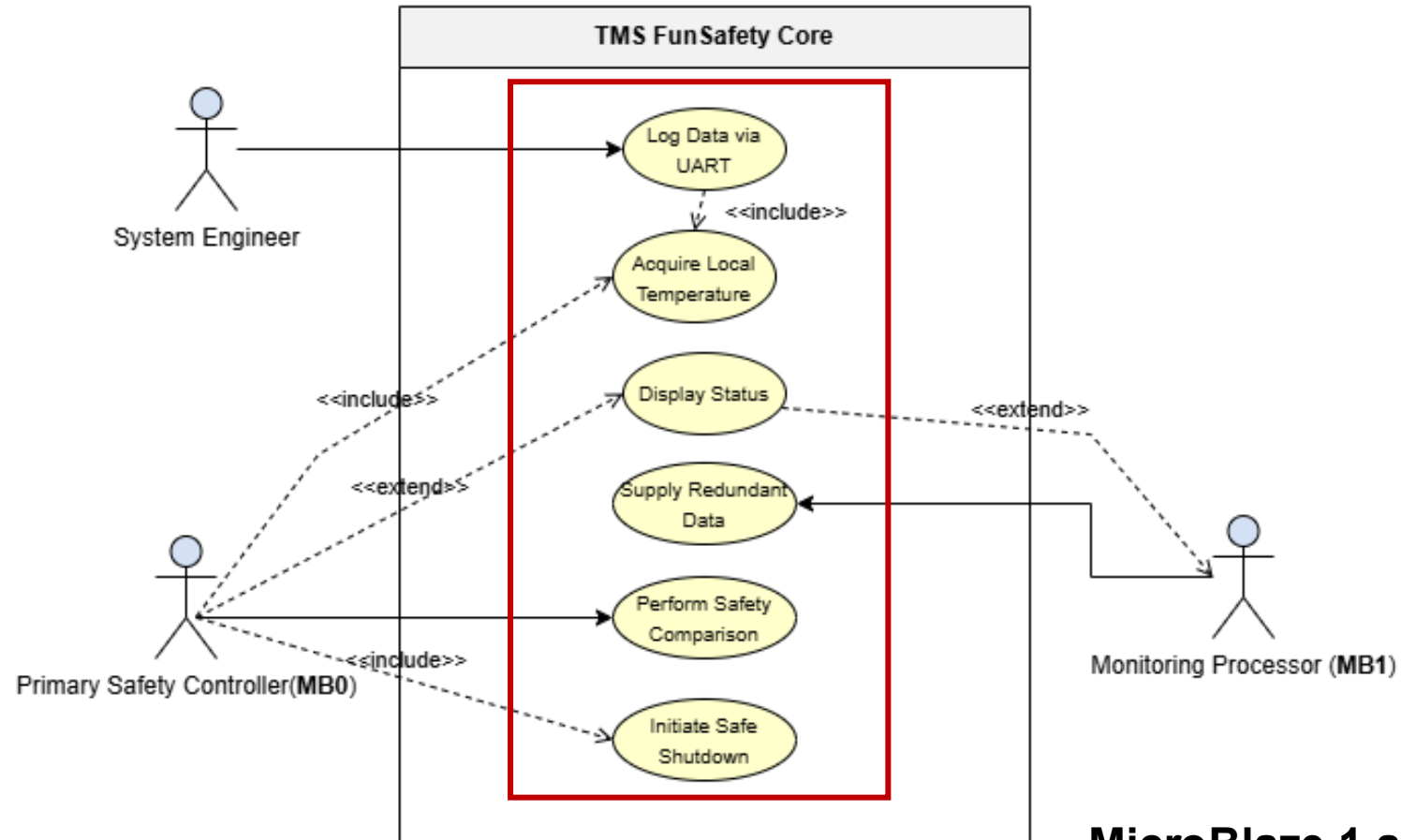
The system is implemented on a Nexys A7 FPGA which hosts two MicroBlaze soft processors running in parallel

Each processor is connected to its own external Pmod TMP3 temperature sensor

Use Case Diagram



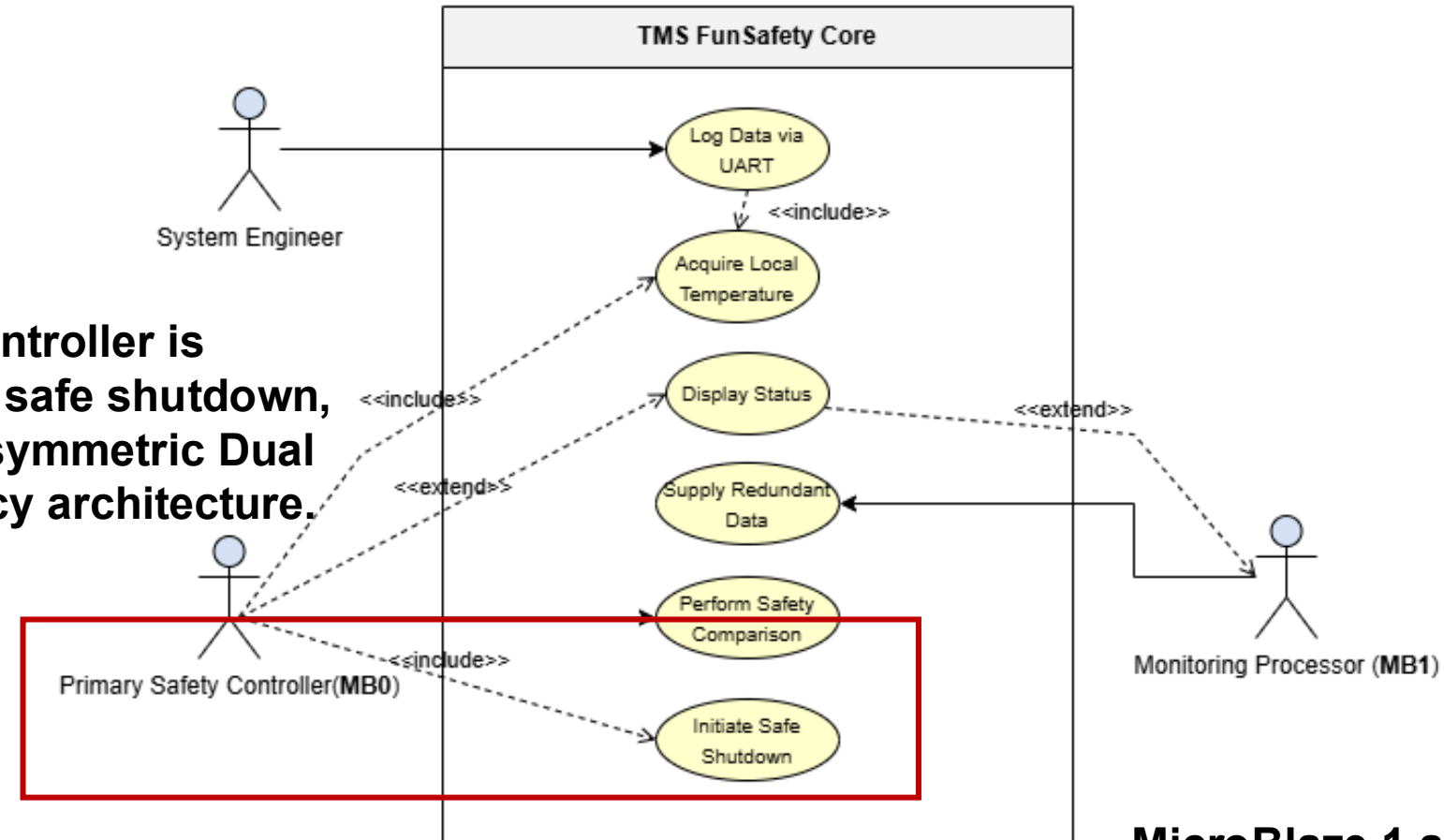
The core safety function is the comparison of the two temperature values



MicroBlaze 0 is the Primary Safety Controller

MicroBlaze 1 acts as a monitoring processor that supplies redundant data

The core safety function is the comparison of the two temperature values

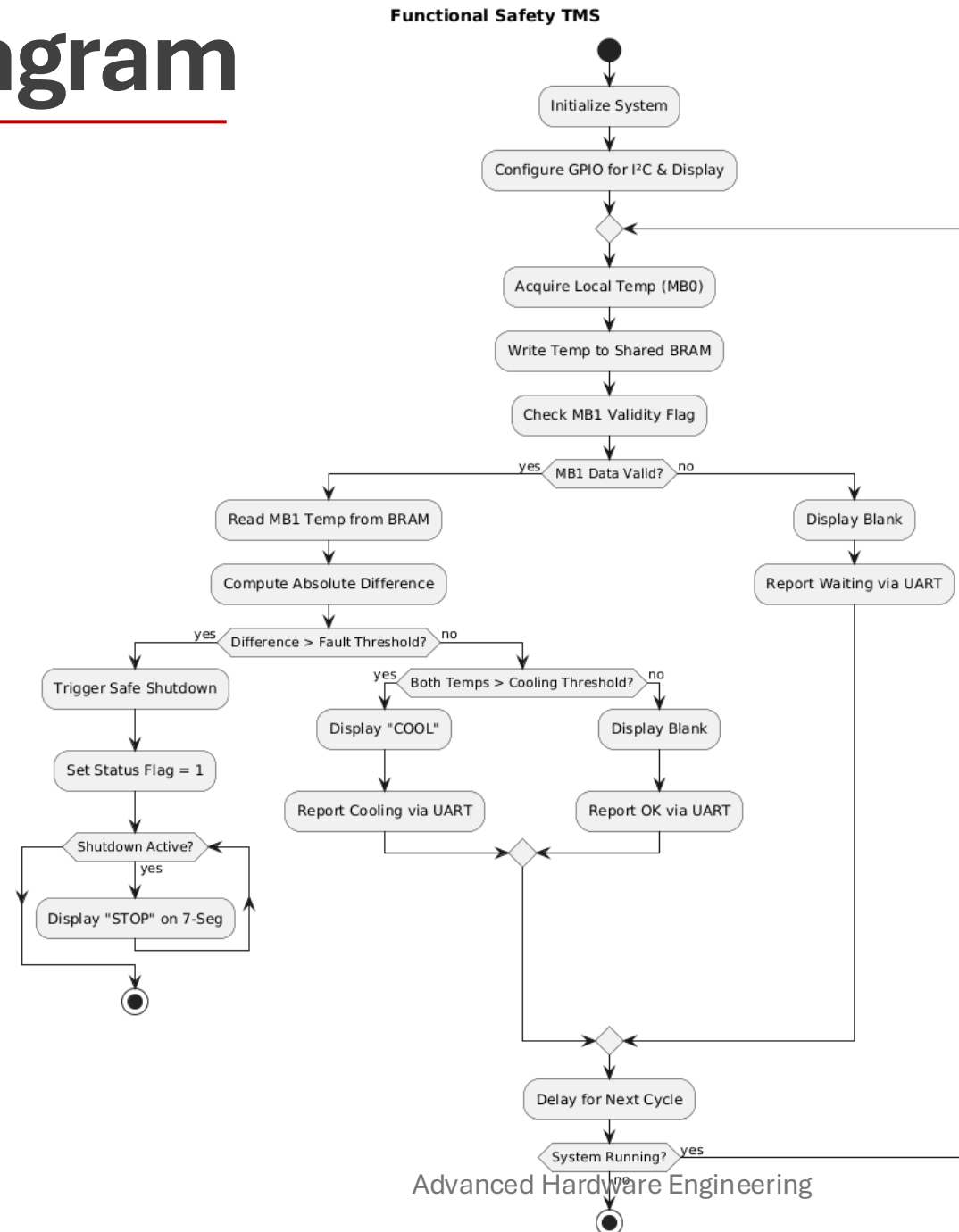


Only the primary controller is allowed to initiate a safe shutdown, which reflects an asymmetric Dual Modular Redundancy architecture.

MicroBlaze 0 is the Primary Safety Controller

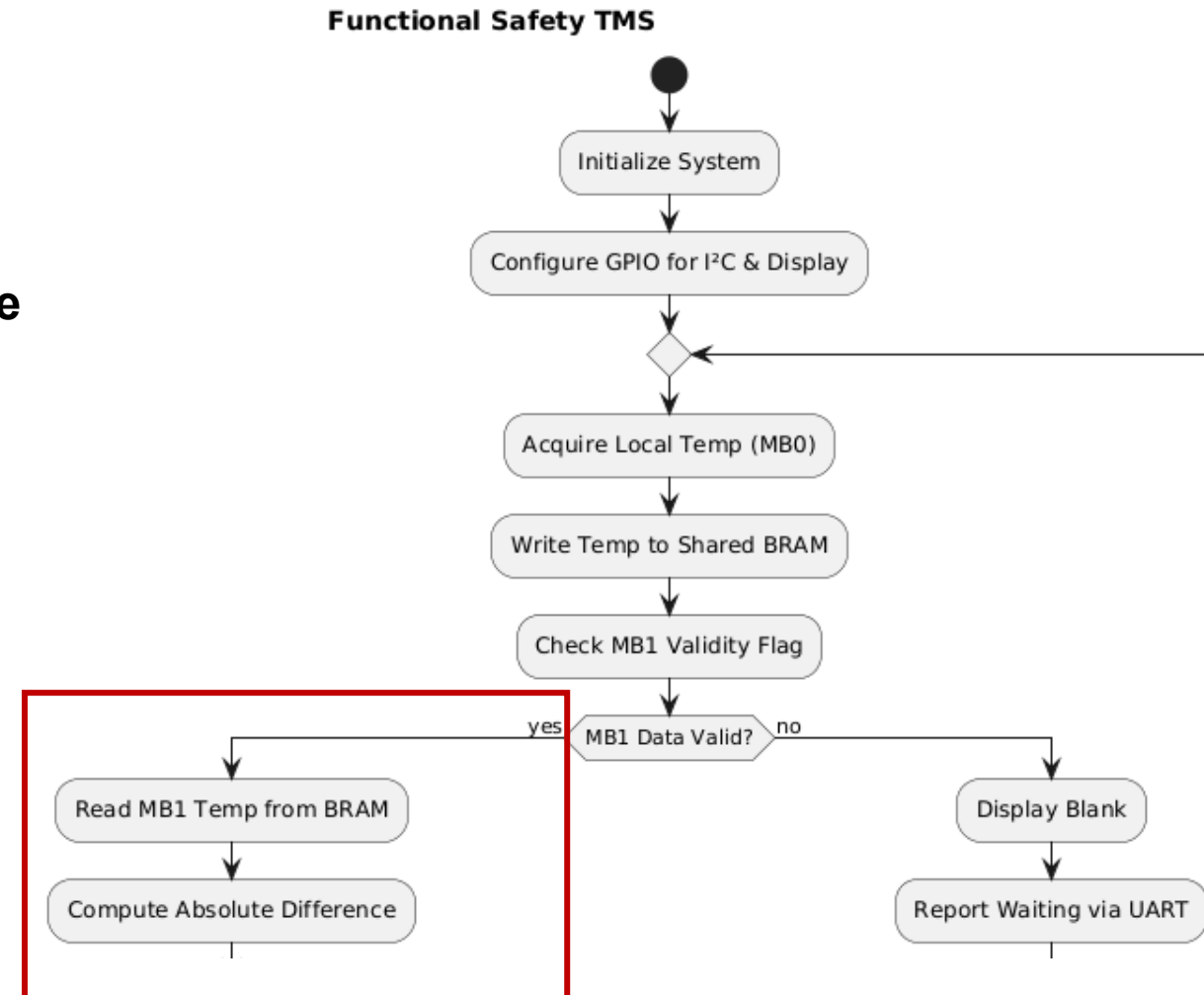
MicroBlaze 1 acts as a monitoring processor that supplies redundant data

Activity Diagram

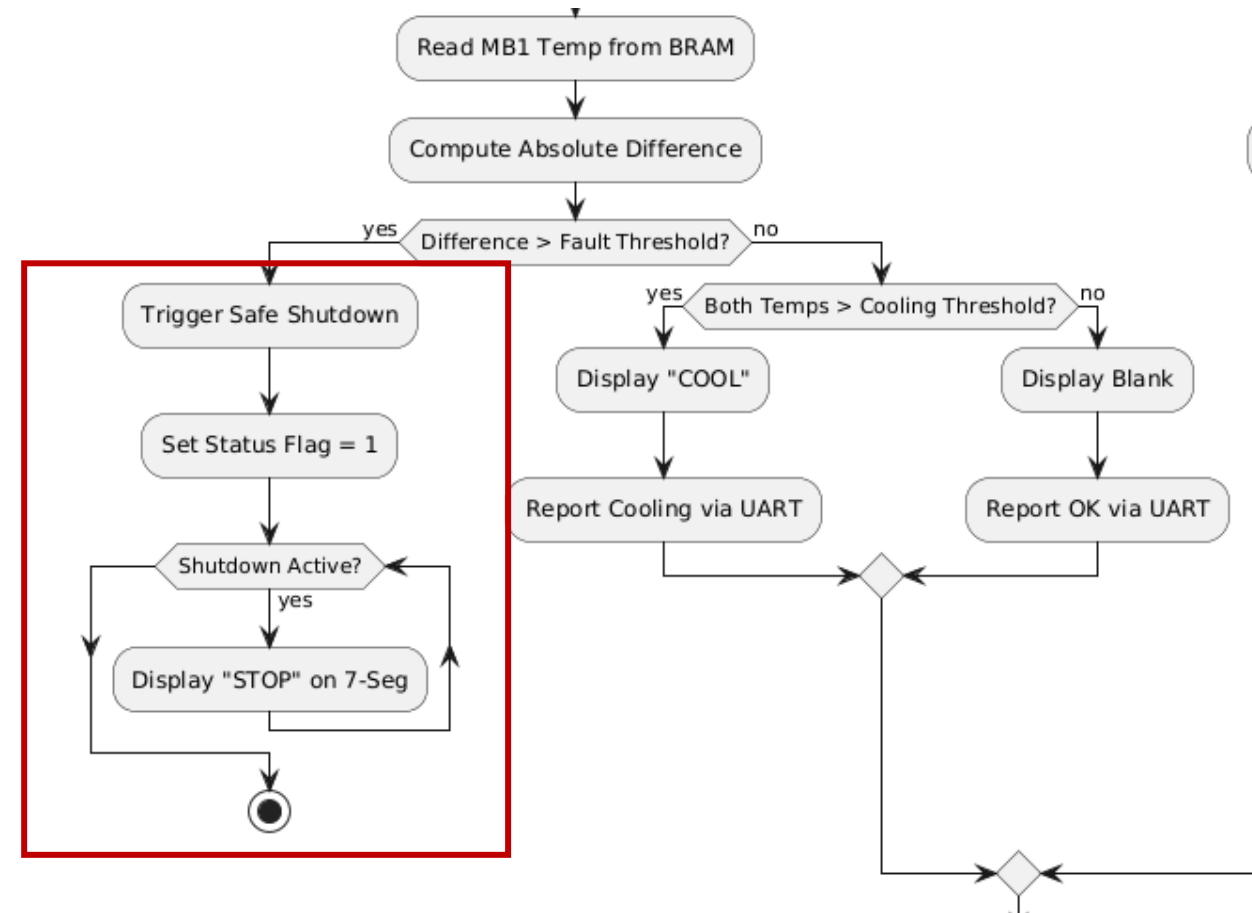
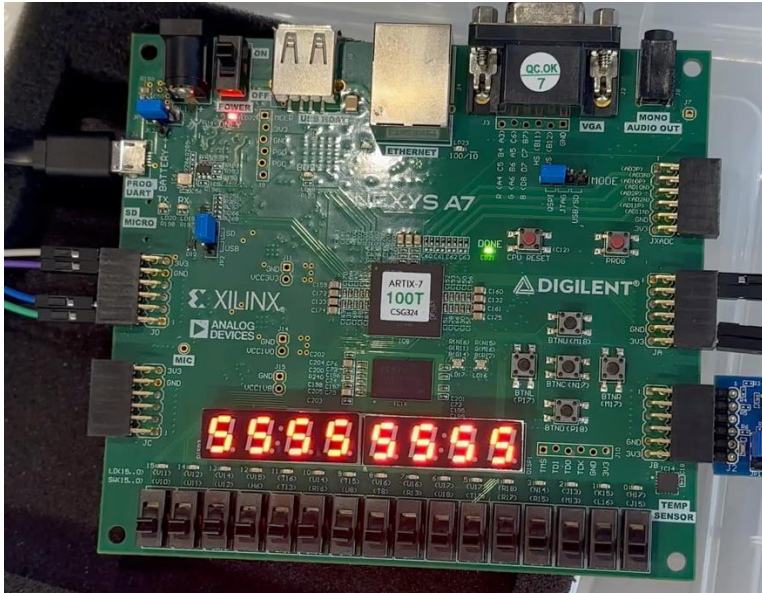


After initialization, the system continuously reads the local temperature and waits for the redundant value

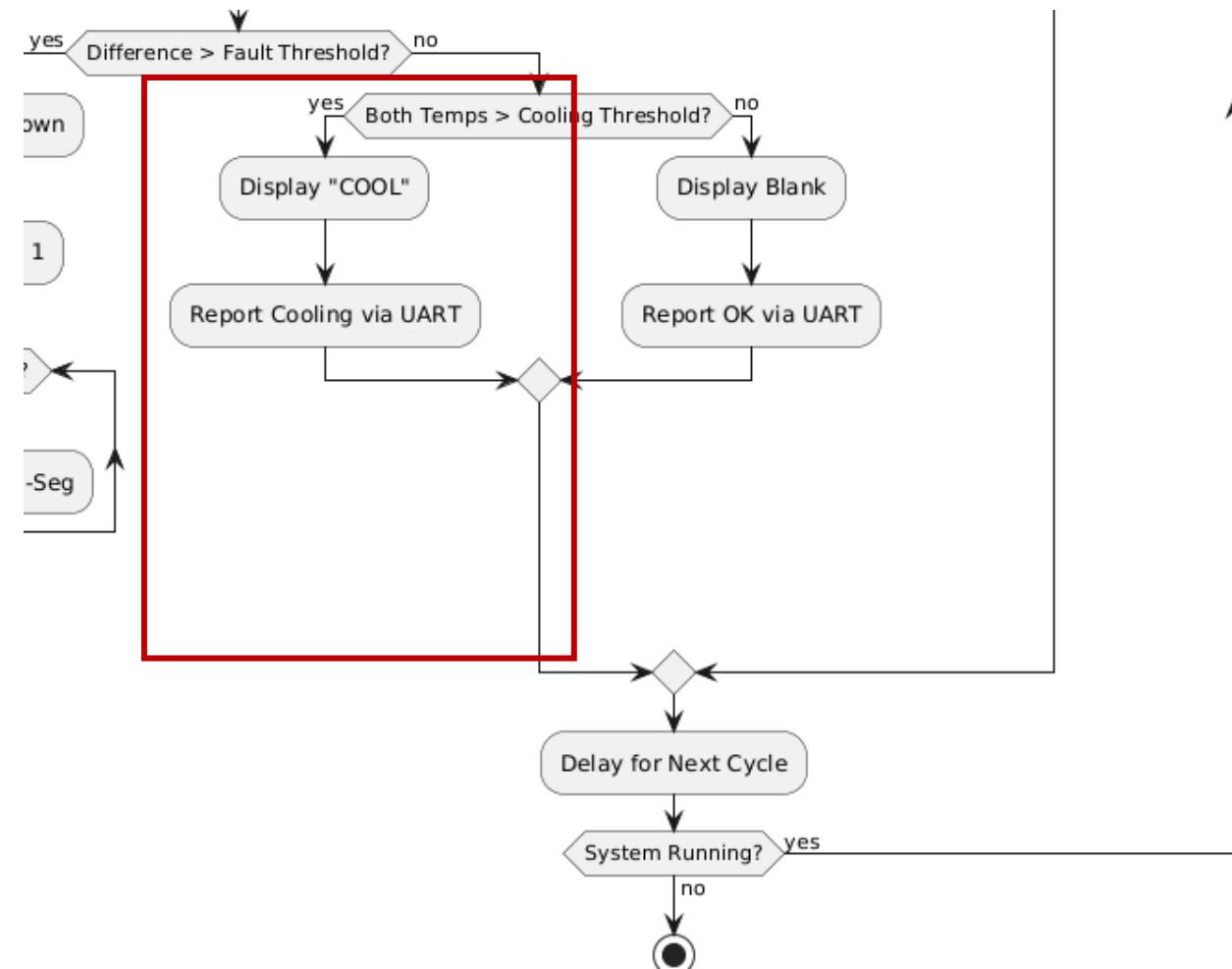
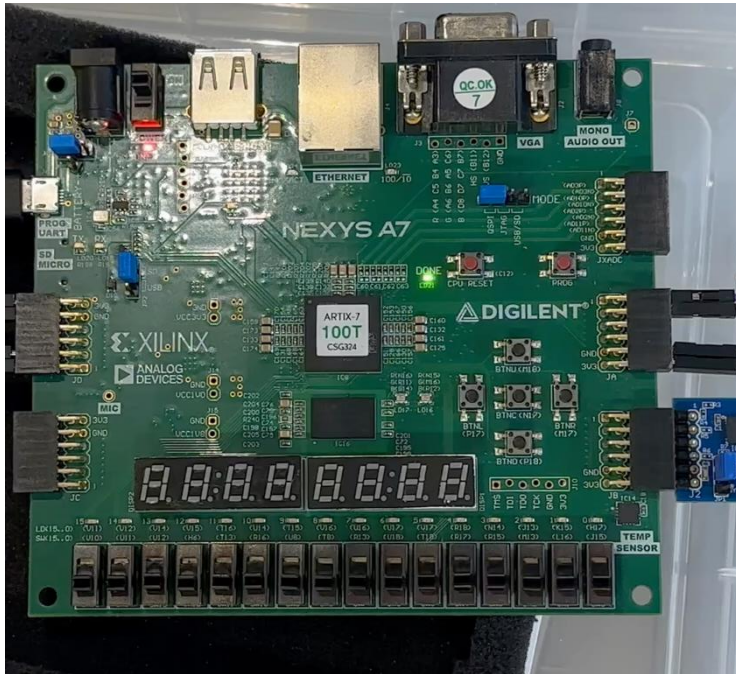
Once both values are available, the system computes the absolute difference



If the difference exceeds a 3 degrees cel, the system immediately enters a **safe shutdown state**

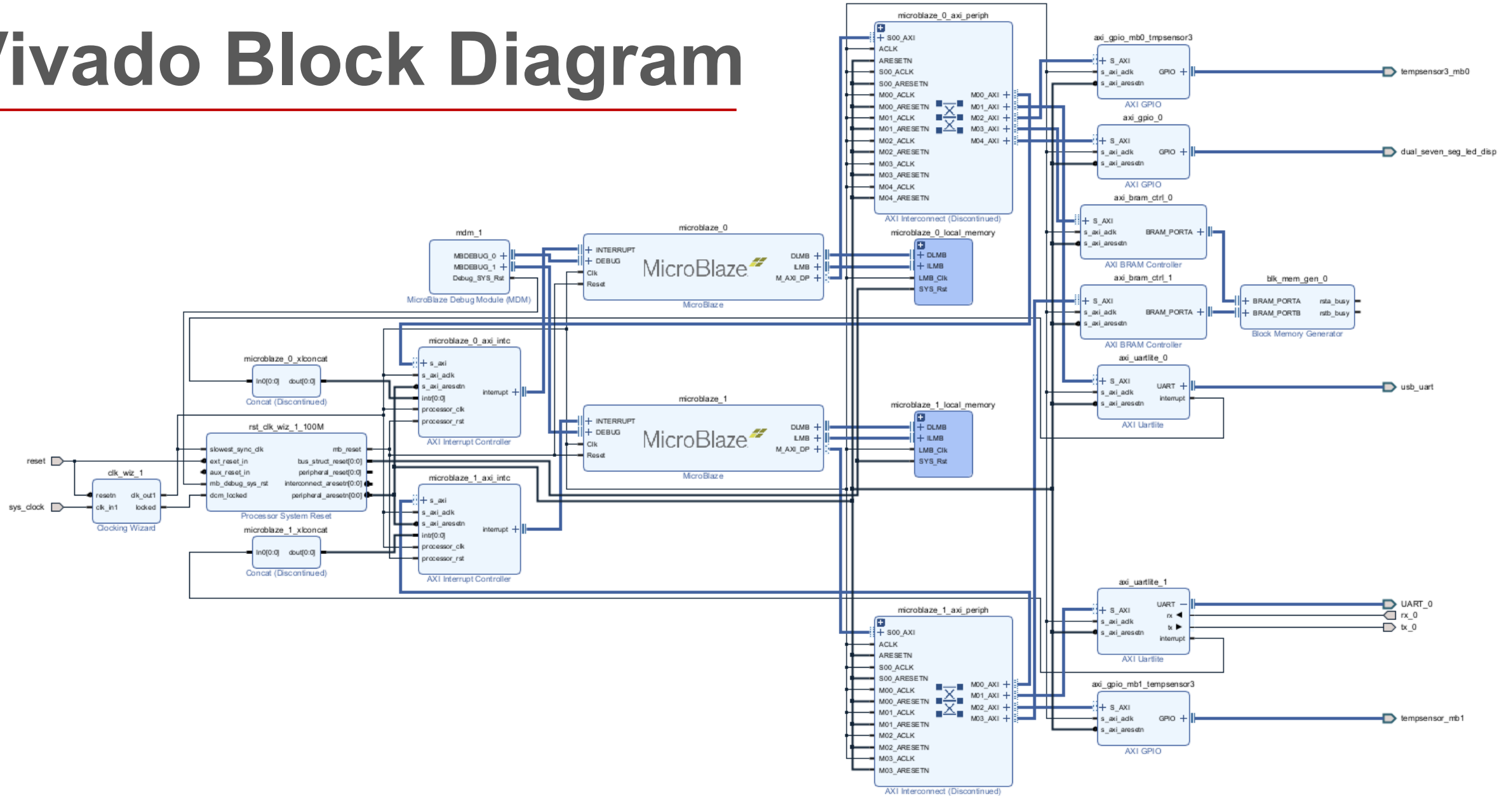


Otherwise, the system either indicates **cooling**



or continues normal operation

Vivado Block Diagram



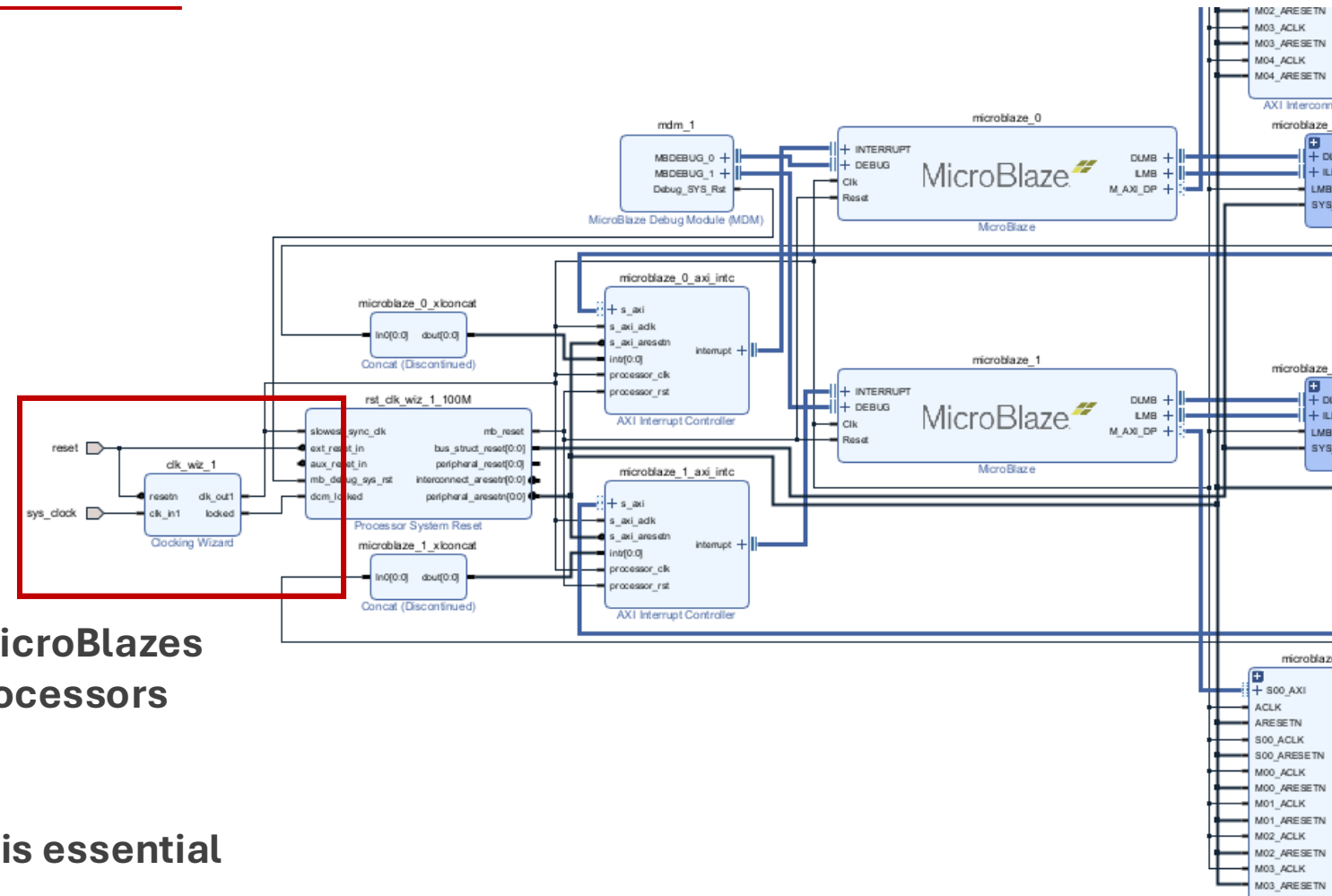
Vivado Block Diagram

One of our main challenges was clock synchronization between the two MicroBlaze processors.

Initially, using separate clock sources caused timing conflicts and prevented successful bitstream generation.

To solve this, we approached the problem systematically and connected both MicroBlazes to a single Clock Wizard, ensuring that both processors operate on the same system clock.

This guarantees deterministic behavior, which is essential for redundancy and safety comparison.

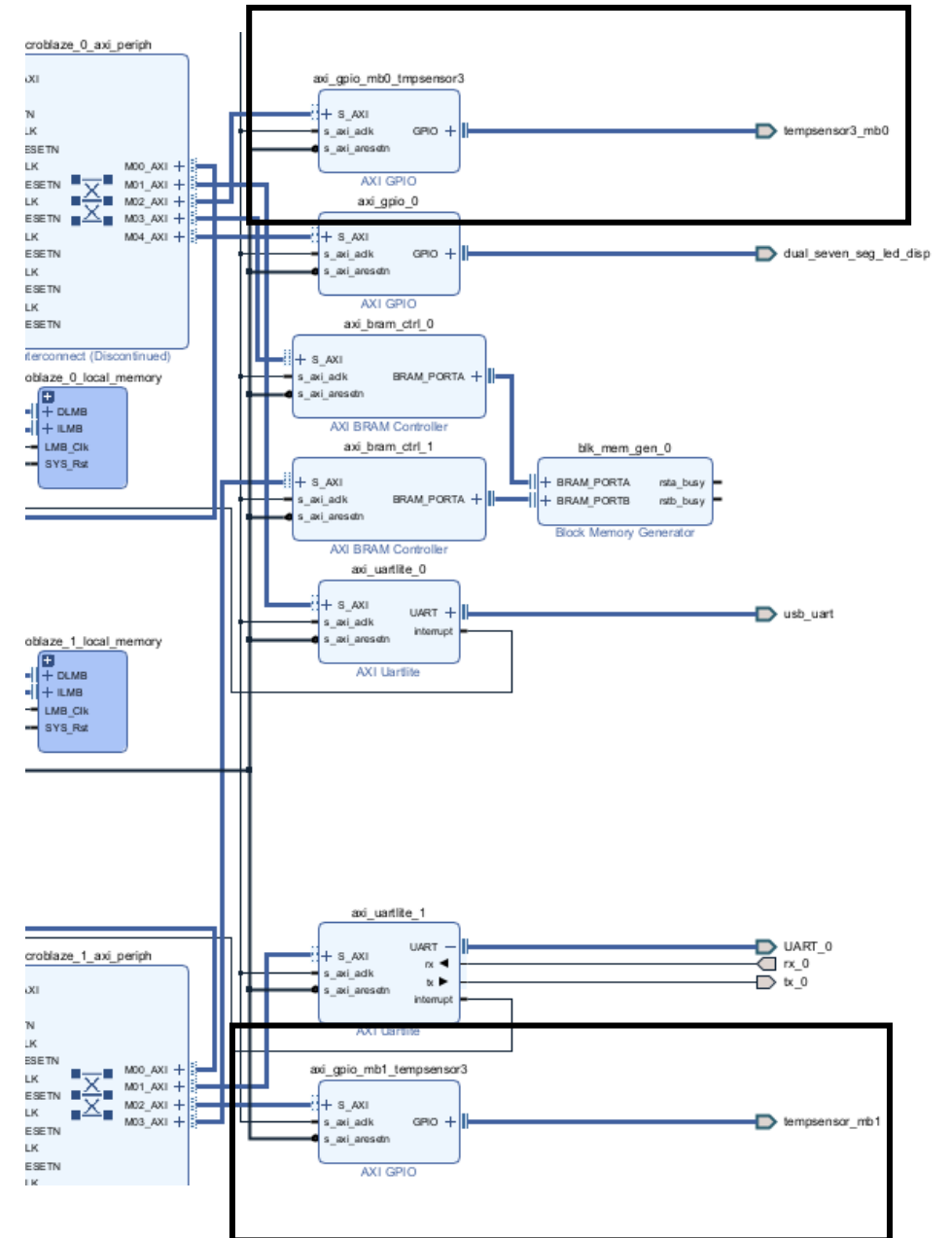


Vivado Block Diagram

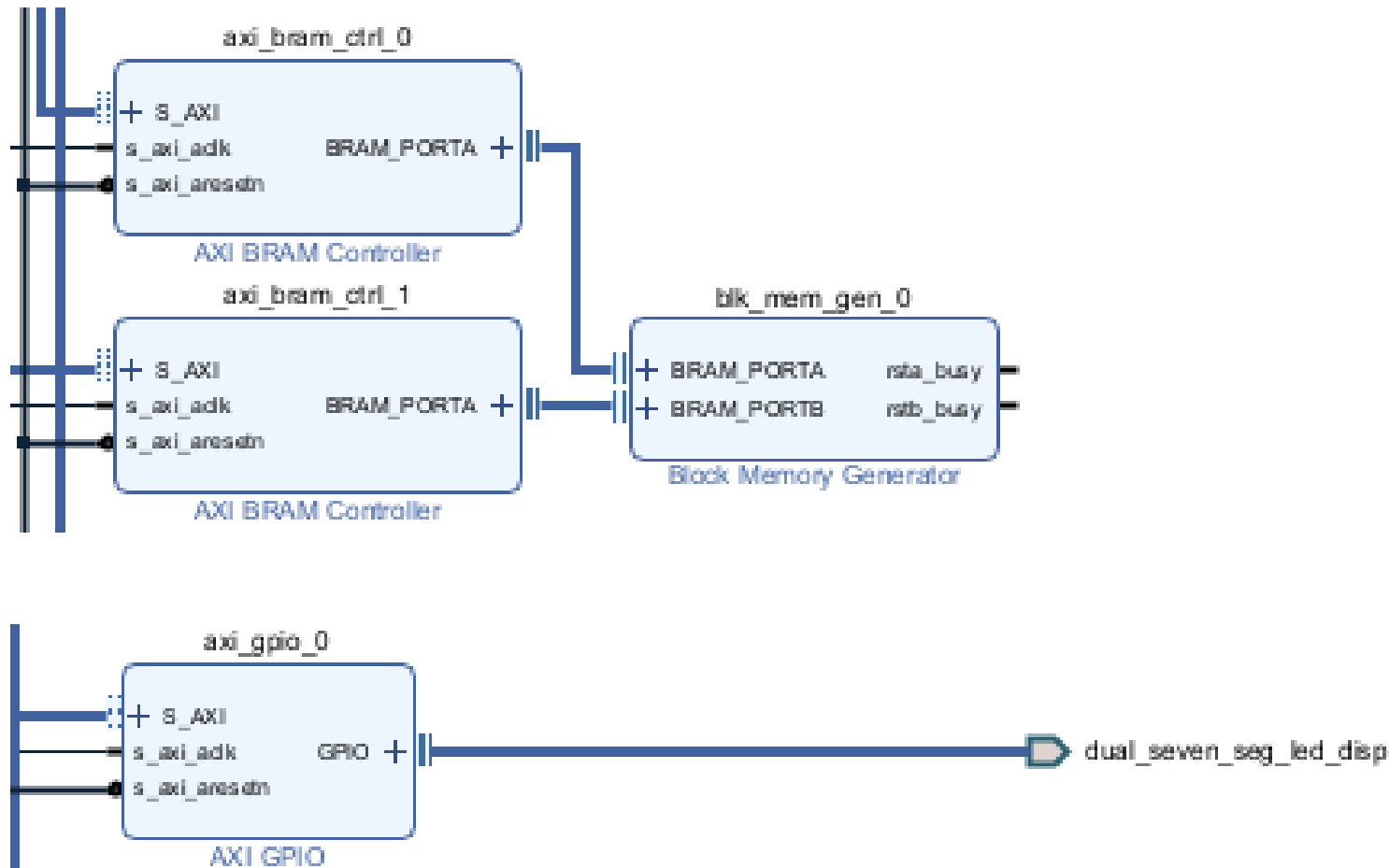
We implemented a bit-banged I²C communication using AXI GPIO.

We configured a GPIO port with two pins, representing the SDA and SCL lines, and implemented the I²C protocol entirely in software.

This allows the MicroBlaze processor to communicate directly with the temperature sensor through the GPIO physical pins, without using a dedicated I²C controller.



Comparator & 7Seg IPs



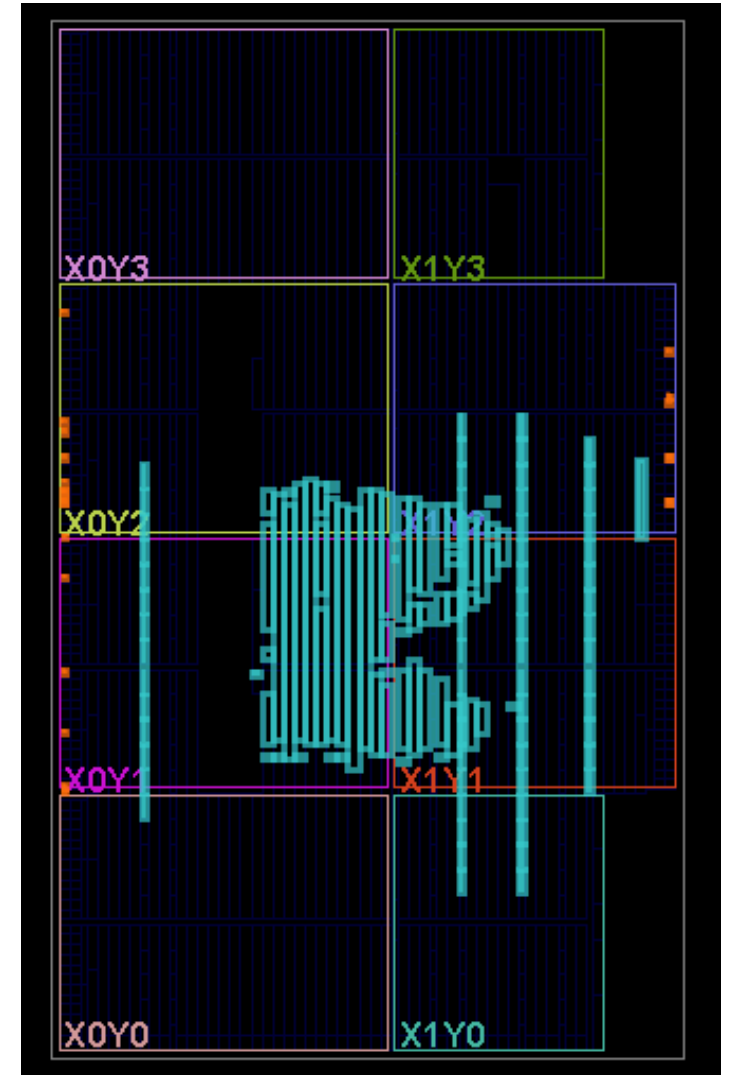
Once communication was established, we implemented a software based comparator using shared Block RAM.

MicroBlaze 1 writes its temperature value into shared memory, and MicroBlaze 0 reads it to perform the safety comparison.

This approach allows flexible threshold control while keeping the hardware design simple.

Comparator & 7Seg IPs

This figure shows the post-synthesis netlist and placement view of our design on the FPGA



Vitis

1 Platform

2 Apps

2 Different Domains

2 UARTs

Challenges with:

`#include "xparameters.h"`

- Search for XPAR

`#include "xgpio.h"`

- Used `xil_io.h`

The screenshot shows the Vitis IDE interface. On the left, the 'VITIS EXPLORER' pane displays the project structure for 'mb0compare'. It includes a 'src' directory with files 'helloworld.c', 'lscript.ld', 'platform.c', and 'platform.h'. Below this, the 'FLOW' pane shows the build process for the 'mb0compare' component, with 'Build' and 'Run' buttons. The main editor displays the source code for 'helloworld.c'. The code includes headers for 'xparameters.h', 'xil_print.h', 'xil_io.h', 'stdint.h', and 'stdlib.h'. It defines shared base addresses for two microblazes (MB0 and MB1) and declares variables for their temperatures, status, and valid addresses. The code also defines constants for fault and cooling temperatures. The output window at the bottom shows the results of the build and execution, displaying the status of each microblaze and the values of the variables.

```
#include "xparameters.h"
#include "xil_print.h"
#include "xil_io.h"
#include <stdint.h>
#include <stdlib.h>

#define SHARED_BASE      XPAR_AXI_BRAM_CTRL_0_BASEADDR
#define TEMP_MB0_ADDR    (SHARED_BASE + 0x00)
#define TEMP_MB1_ADDR    (SHARED_BASE + 0x04)
#define STATUS_ADDR      (SHARED_BASE + 0x08)
#define MB1_VALID_ADDR   (SHARED_BASE + 0x0C)

volatile int32_t  *temp_mb0  = (int32_t *)TEMP_MB0_ADDR;
volatile int32_t  *temp_mb1  = (int32_t *)TEMP_MB1_ADDR;
volatile uint32_t *status    = (uint32_t *)STATUS_ADDR;
volatile uint32_t *mb1_valid = (uint32_t *)MB1_VALID_ADDR;

#define DIFF_FAULT_CENTI 300
#define COOLING_TEMP_CENTI 2500

#ifdef XPAR_AXI_GPIO_MB0_TMPSENSOR3_BASEADDR
#define TMP3_GPIO_BASE    XPAR_AXI_GPIO_MB0_TMPSENSOR3_BASEADDR
#elif defined(XPAR_XGPIO_1_BASEADDR)
#define TMP3_GPIO_BASE    XPAR_XGPIO_1_BASEADDR
#else
#error "Cannot find TMP3 GPIO base address in xparameters.h"
```

OUTPUT × COM8 (FTDI) × COM6 (FTDI) × TASK: XSDB CONSOLE DEBUG CONSOLE × PROBLEMS 2 ×

MB0	MB1	TEMP_MB0	TEMP_MB1	STATUS	MB1_VALID
OK	OK	25.50 C	24.50 C	1.00 C	
OK	OK	25.50 C	24.50 C	1.00 C	
OK	OK	25.50 C	24.50 C	1.00 C	
OK	OK	25.00 C	24.50 C	0.50 C	
OK	OK	25.50 C	24.50 C	1.00 C	
OK	OK	25.50 C	24.50 C	1.00 C	
OK	OK	26.00 C	24.50 C	1.50 C	
OK	OK	26.00 C	24.50 C	1.50 C	
OK	OK	26.50 C	24.50 C	2.00 C	
OK	OK	26.50 C	24.50 C	2.00 C	
OK	OK	26.50 C	24.50 C	2.00 C	
OK	OK	27.00 C	24.50 C	2.50 C	
OK	OK	27.00 C	24.50 C	2.50 C	
OK	OK	27.00 C	24.50 C	2.50 C	

MicroBlaze_0 &1 Codes

```
1 static int sda_read(void)
2 {
3     set_dir(SDA_MASK, 1);
4     return (tmp3_data_read() & SDA_MASK) ? 1 : 0;
5 }
6
7 static void i2c_init(void)
8 {
9     tmp3_data_write(0x0);
10    set_dir(SCL_MASK | SDA_MASK, 1);
11 }
12
13 static void i2c_start(void)
14 {
15     sda_write(1); scl_write(1); i2c_delay();
16     sda_write(0); i2c_delay();
17     scl_write(0);
18 }
19
20 static void i2c_stop(void)
21 {
22     sda_write(0); i2c_delay();
23     scl_write(1); i2c_delay();
24     sda_write(1); i2c_delay();
25 }
26
27 static int i2c_write_byte(uint8_t data)
28 {
29     for (int i = 7; i >= 0; --i) {
30         sda_write((data >> i) & 1);
31         i2c_delay();
32         scl_write(1); i2c_delay();
33         scl_write(0);
34     }
35
36     sda_write(1);
37     i2c_delay();
38     scl_write(1); i2c_delay();
39     int ack = !sda_read();
40     scl_write(0);
41     return ack;
42 }
```

```
1 if (*mb1_valid == 0) {
2     xil_printf("MB0: MB0=%d.%02d C (waiting for MB1)\r\n", deg0, cent0);
3     seg_write(SEG_BLANK);
4     delay_cycles(8000000);
5     continue;
6 }
```

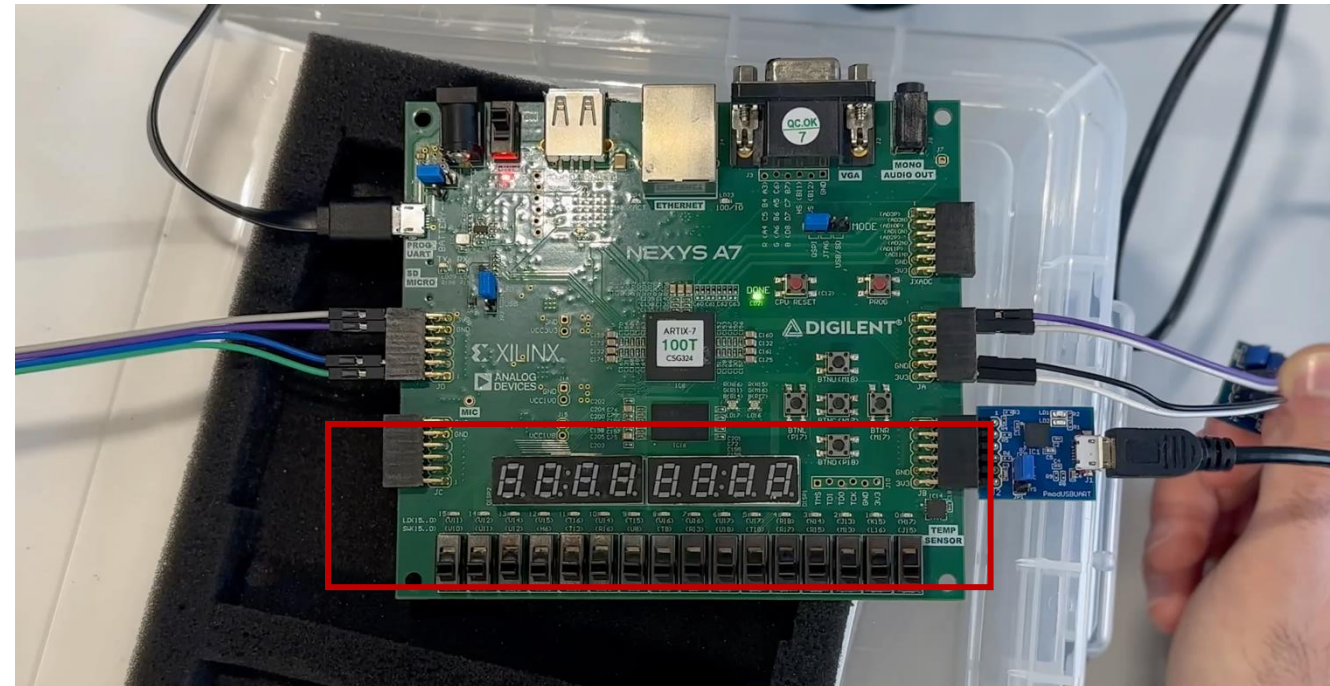
```
if (diff >= DIFF_FAULT_CENTI) {
    *status = 1;
    xil_printf("SHUTDOWN\r\n");
    *temp_mb0 = t0_centi;
    while (1) {
        seg_show_sequence(MSG_STOP, 4);
    }

    if ((t0_centi > COOLING_TEMP_CENTI) && (t1_centi > COOLING_TEMP_CENTI)) {
        *status = 0;
        xil_printf("comparator: COOL MB0=%d.%02d C, MB1=%ld.%02ld C\r\n",
            deg0, cent0, (long)deg1, (long)cent1);
        seg_show_sequence(MSG_COOL, 4);
    } else {
        *status = 0;
        xil_printf("MB0: OK MB0=%d.%02d C, MB1=%ld.%02ld C, |Δ|=%ld.%02ld C\r\n",
            deg0, cent0, (long)deg1, (long)cent1,
            (long)(diff/100), (long)abs(diff%100));
        seg_write(SEG_BLANK);
        delay_cycles(8000000);
    }
}
```

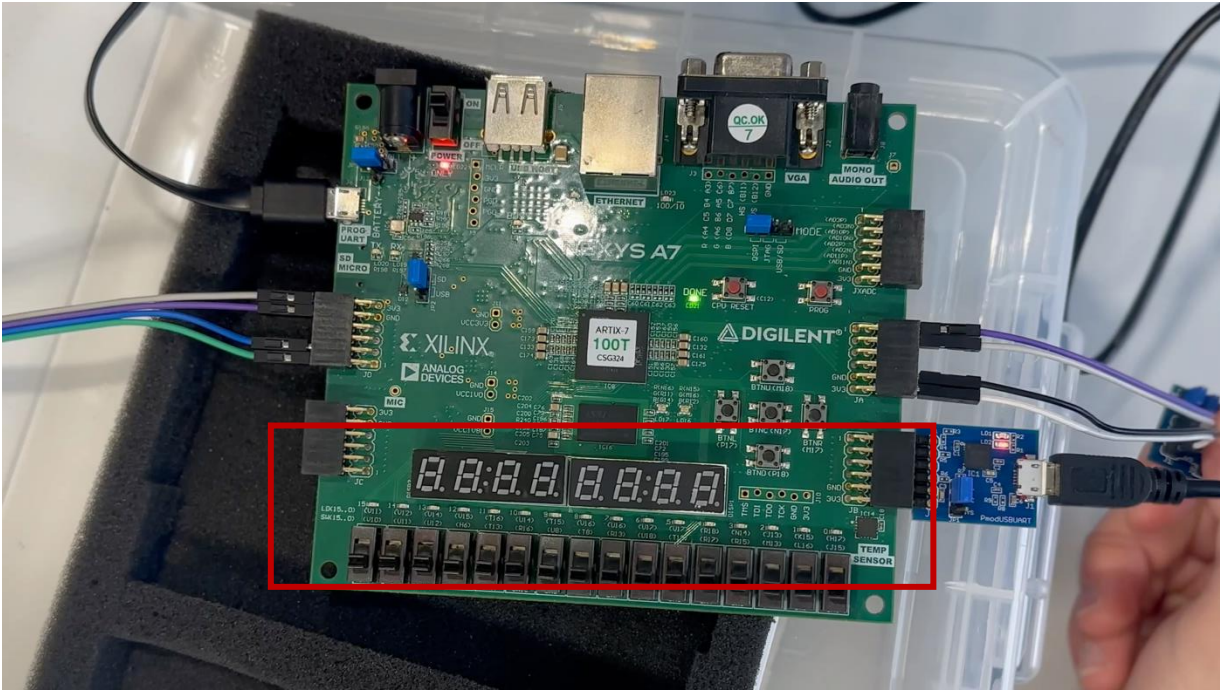
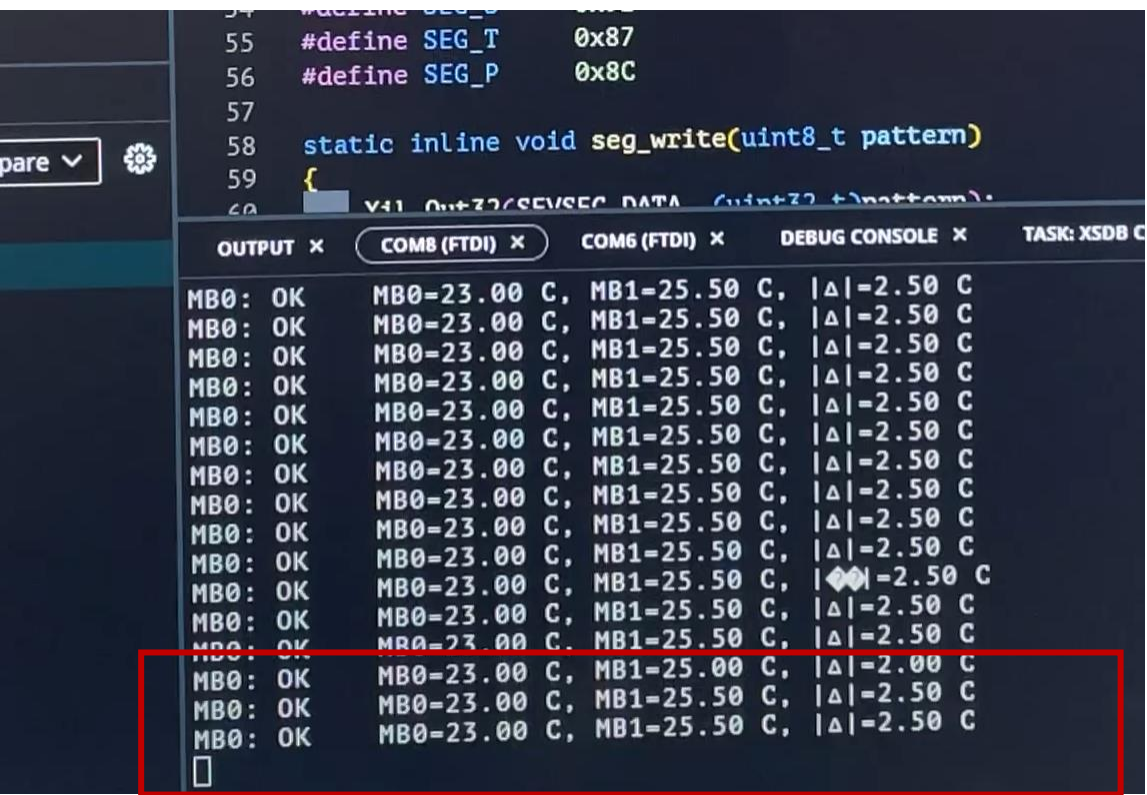

Live Video COOLING case

```
20
21 #ifdef XPAR_AXI_GPIO_MB0_TMPSENSOR3_BASEADDR
22     #define TMP3_GPIO_BASE    XPAR_AXI_GPIO_MB0_TMPSENSOR3_BASEADDR
23 #elif defined(XPAR_XGPIO_1_BASEADDR)
24     #define TMP3_GPIO_BASE    XPAR_XGPIO_1_BASEADDR
25 #else
26     #error "Cannot find TMP3 GPIO base address in xparameters.h"
```

OUTPUT	COM8 (FTDI)	COM6 (FTDI)	TASK: XSD8 CONSOLE	DEBUG CONSOLE	PROBLEM
MB0: OK	MB0-22.50 C,	MB1-22.00 C,	Δ =-0.50 C		
MB0: OK	MB0-22.50 C,	MB1-22.00 C,	Δ =-0.50 C		
MB0: OK	MB0-23.00 C,	MB1-22.50 C,	Δ =-0.50 C		
MB0: OK	MB0-23.00 C,	MB1-22.50 C,	Δ =-0.50 C		
MB0: OK	MB0-23.50 C,	MB1-22.50 C,	Δ =-1.00 C		
MB0: OK	MB0-24.00 C,	MB1-23.00 C,	Δ =-1.00 C		
MB0: OK	MB0-24.00 C,	MB1-23.00 C,	Δ =-1.00 C		
MB0: OK	MB0-24.50 C,	MB1-23.50 C,	Δ =-1.00 C		
MB0: OK	MB0-24.50 C,	MB1-23.50 C,	Δ =-1.00 C		
MB0: OK	MB0-25.00 C,	MB1-24.00 C,	Δ =-1.00 C		
MB0: OK	MB0-25.00 C,	MB1-24.00 C,	Δ =-1.00 C		
MB0: OK	MB0-25.00 C,	MB1-24.00 C,	Δ =-1.00 C		
MB0: OK	MB0-25.50 C,	MB1-24.50 C,	Δ =-1.00 C		
MB0: OK	MB0-25.50 C,	MB1-24.50 C,	Δ =-1.00 C		
MB0: OK	MB0-25.50 C,	MB1-24.50 C,	Δ =-1.00 C		
MB0: OK	MB0-26.00 C,	MB1-24.50 C,	Δ =-1.50 C		
MB0: OK	MB0-26.00 C,	MB1-25.00 C,	Δ =-1.00 C		



Live Video COOLING case



Thank You for your Attention
Happy to hear your Questions

